

Artificial Intelligence Laboratory: Search and A*

Using an LLM as an Engineering Assistant

Warehouse Robot Navigation

1. Task 0 — Formulation of the Search Problem

The warehouse map supplied in the laboratory is:

```
#####  
#S...#.....#  
#.###.#.#####.#  
#...#.#.....#.#  
###.#.#####.#.#  
#...#.....#.#  
#.#####.#.#  
#.....#G#  
#####
```

Component	Specification
State S	Robot's (row, column) position.
Actions A	Up, Down, Left, Right.
Transition T	Move one cell in the selected direction if inside the grid and not an obstacle.
Initial state s_0	Position containing S.
Goal G	Position containing G.
Cost c	1 for every movement.

(a) What information is necessary to specify a state?

The robot's current row and column are sufficient because the warehouse layout is fixed.

(b) What makes an action invalid?

Moving outside the grid or into a cell containing #.

(c) Is this a deterministic search problem?

Yes. A state and valid action uniquely determine the successor state.

(d) What would constitute a solution?

A sequence of valid moves from S to G that never enters an obstacle.

2. Task 1 — Plan the Agent

Component	Design
State representation	Tuple (row, column).
Warehouse representation	List of strings using #, ., S, and G.
Valid actions	Generate the four neighbouring cells and keep only in-bounds non-obstacle cells.
Goal test	Current state equals the goal coordinate.
Frontier	A min-priority queue containing $f(n)$, $g(n)$, a tie-break counter, and the state.

Path reconstruction	Parent pointers plus the action used to reach each state.
---------------------	---

The program reports solution status, the path, path length, and number of states expanded.

3. Task 2 — LLM Prompt Engineering

Exact prompt used:

I am implementing a simple goal-based search agent in Python for the following warehouse.

```
#####
#S...#.....#
#...#.#.....#
#...#.#.....#
#...#.#.....#
#...#.#.....#
#...#.#.....#
#...#.#.....#
#.....#G#
#####
```

*The robot starts at S and must reach G. # is an obstacle and . is free.
It can move Up, Down, Left, or Right, and every movement costs 1.*

Implement A search using Manhattan distance:
 $h(n) = |x - x_G| + |y - y_G|$.*

Requirements:

- represent grid positions as states;
- maintain a priority-queue frontier;
- calculate $g(n)$, $h(n)$, and $f(n)=g(n)+h(n)$;
- avoid repeatedly expanding the same state;
- reconstruct the path when the goal is reached;
- report whether a solution was found, the path, path length, and number of states expanded;
- keep the implementation self-contained in standard Python 3.

Also structure the code so that the heuristic can easily be replaced with zero, Euclidean distance, or twice the Manhattan distance for later experiments.

The generated implementation was inspected against the design before execution. The final implementation explicitly maintains $g(n)$, computes $h(n)$, forms $f(n)=g(n)+h(n)$, uses a priority queue, prevents repeated expansion with a closed set, and reconstructs the path through parent pointers.

4. Task 3 — Systematic Testing

Test	Result	Observed behaviour
------	--------	--------------------

Original warehouse	PASS	Solution found; path length 40; 64 states expanded.
Trivial adjacent goal	PASS	One-step solution found.
No solution	PASS	Search terminates with failure; 9 states expanded.
Alternative paths	PASS	Shortest path of length 4 found.

4.1 Original warehouse result

Solution found: True

Path length: 40 moves

States expanded: 64

Path:

(1, 1) → (1, 2) → (1, 3) → (1, 4) → (1, 5) → (2, 5) → (3, 5) → (4, 5) → (5, 5) → (5, 6) → (5, 7) → (5, 8) → (5, 9) → (5, 10) → (5, 11) → (5, 12) → (5, 13) → (4, 13) → (3, 13) → (3, 12) → (3, 11) → (3, 10) → (3, 9) → (3, 8) → (3, 7) → (2, 7) → (1, 7) → (1, 8) → (1, 9) → (1, 10) → (1, 11) → (1, 12) → (1, 13) → (1, 14) → (1, 15) → (2, 15) → (3, 15) → (4, 15) → (5, 15) → (6, 15) → (7, 15)

Actions:

Right → Right → Right → Right → Down → Down → Down → Down → Right → Right → Right → Right → Right → Right → Up → Up → Left → Left → Left → Left → Left → Left → Up → Up → Right → Right → Right → Right → Right → Right → Right → Right → Right → Down → Down → Down → Down → Down

5. Task 4 — Inspect the A* Algorithm

Concept	Where it appears
State	(row, column) tuple.
Action	ACTIONS dictionary.
Transition	valid_neighbors().
Goal test	current == goal.
$g(n)$	g dictionary storing best known start cost.
$h(n)$	heuristic() function.
$f(n)$	tentative_g + heuristic(...).
Frontier	heapq priority queue.
Visited states	closed set.
Path reconstruction	reconstruct().

(a) What data structure is used for the A* frontier?

A min-priority queue implemented with Python's heapq.

(b) How does the program select the next state to expand?

It removes the state with the smallest $f(n)=g(n)+h(n)$.

(c) Where is the heuristic calculated?

In heuristic(), when successor states are inserted into the frontier.

(d) Does the program explicitly calculate $f(n)=g(n)+h(n)$?

Yes: `fscore = tentative_g + heuristic(...)`.

(e) How does the program prevent unnecessary repeated exploration?

The closed set prevents already-expanded states from being expanded again, while `g` stores the best known cost and rejects non-improving routes.

6. Task 5 — Compare A* with Blind Search

Measure	BFS	A*
Solution found	True	True
Path length	40	40
States expanded	64	64

Both algorithms found a solution and returned paths of the same length. In this supplied warehouse, both expanded 64 states. This particular map forms a constrained corridor-like route, so the Manhattan heuristic does not reduce the number of expansions relative to BFS. In a less constrained map, an informative heuristic can direct A* toward the goal and reduce unnecessary exploration.

7. Task 6 — Investigate the Heuristic

Manhattan distance is appropriate because movement is restricted to horizontal and vertical grid steps. The value $|x-x_G|+|y-y_G|$ is the minimum number of moves needed if obstacles are ignored. Obstacles cannot make the actual remaining path shorter, so this is an admissible lower bound.

Heuristic	Solution found	Path length	States expanded
Manhattan	True	40	64
$h(n)=0$	True	40	64
Euclidean	True	40	64
$2 \times \text{Manhattan}$	True	40	64

The zero heuristic removes goal-direction information and reduces A* to uniform-cost search; because every movement costs 1, its behaviour here matches BFS. Euclidean distance is also a lower bound for four-direction movement, although Manhattan distance better matches the actual movement metric. Doubling Manhattan makes the heuristic more aggressive and generally inadmissible because it can exceed the true remaining cost. On this specific map it still returned the same 40-move solution, but optimality is not guaranteed in general once admissibility is lost.

8. Task 7 — Evaluate the LLM-Generated Agent

1. What parts of the generated code were correct immediately?

The core A* structure was correct: coordinate states, four movement actions, a priority-queue frontier, `g/h/f` calculations, parent pointers, and state-expansion reporting.

2. Did you find any bugs or design problems?

No correctness bug was found in the final implementation. The important design point was to make repeated-state handling explicit using both best-known `g` costs and a closed set.

3. How did you discover those problems?

By checking the implementation against the search specification and running the original, trivial, no-solution, and alternative-path tests.

4. Did the LLM use terminology or data structures that you did not understand?

The main terms requiring interpretation were frontier, priority queue, $g(n)$, $h(n)$, $f(n)$, and closed states. Each was mapped to a specific part of the implementation.

5. Did you modify the LLM-generated code?

Yes. The search implementation was structured so the heuristic could be replaced for the required experiments, and a BFS implementation was added for direct comparison.

6. Which tests were most useful?

The no-solution and alternative-path tests were especially useful because they test termination/failure handling and path-quality behaviour rather than merely producing a plausible route.

7. Could you have trusted the program without testing it?

No. Search code can produce plausible output while mishandling obstacles, repeated states, unreachable goals, or shortest-path guarantees.

8. What did you understand about A* that you did not understand before implementing it?

The implementation makes the role of the heuristic concrete: A* ranks frontier states using both the cost already incurred and an estimate of the remaining cost. Changing h changes the search order and potentially the amount of exploration.

9. Final Reflection

1. Why is it important to formulate the search problem before writing the search algorithm?

A precise formulation defines states, actions, transitions, start, goals, and costs. Without it, code can be correct syntactically but solve the wrong problem. Here, the formulation also explains why BFS and A* fit the equal-cost grid.

2. In what sense is A* an “informed” search algorithm?

A* uses problem-specific information about the estimated remaining distance to the goal. It combines $g(n)$, the cost already incurred, with $h(n)$, the estimated remaining cost, to form $f(n)$. BFS does not use this goal-distance estimate.

3. Why does the choice of heuristic matter?

The heuristic determines search order. An admissible heuristic can guide A* while retaining the shortest-path guarantee. $h=0$ removes guidance, while an aggressive heuristic such as $2 \times \text{Manhattan}$ can lose admissibility and therefore the general optimality guarantee.

4. What did the LLM contribute to the engineering process?

It translated the specified search design into Python boilerplate and suggested the priority-queue and path-reconstruction structure. This reduced implementation effort while leaving specification, testing, and validation to the engineer.

5. What could go wrong if an engineer simply accepted LLM-generated code without testing it?

The program could use incorrect transitions, mishandle obstacles, fail on unreachable goals, reconstruct invalid paths, or return non-shortest paths. A plausible output is not sufficient evidence that the algorithm is correct.